Engineering Exercise

Distributed Log Ingestion and Processing Pipeline

1. Overview

We would like you to design and implement a small but production-minded log ingestion pipeline. The goal is not feature breadth, it is to see how you think about reliability, backpressure, component boundaries, and code quality under realistic constraints. Treat this as a scaled-down version of the kind of pipeline you would propose for a real customer environment and make the trade-offs explicit. We are far more interested in the quality of a small system than in the size of a large one.

2. Problem Context

Imagine an environment with a few hundred network devices and application hosts emitting logs. Operators need to search recent events by host, severity, and free text; downstream services need a reliable stream they can subscribe to; and the platform team needs the pipeline itself to be observable and graceful under partial failure (storage slow, network blips, malformed input).

You will build the three core stages of such a pipeline, collection, transport, and processing/storage, plus the operational glue around them.

3. Functional Requirements

3.1 Collector (Agent)

Implement a collector capable of ingesting events from at least two of the following sources (your choice, pick what best demonstrates your design):

- Tailing one or more files on disk, including handling file rotation (rename/create) and partial last-line writes.
- RFC 5424 syslog over UDP and/or TCP.
- HTTP endpoint accepting JSON or NDJSON batches.

The collector must:

- Support multi-line events (e.g., stack traces) with a configurable continuation rule.
- Buffer locally so a brief transport outage does not lose events. Be explicit about your durability guarantees (in-memory, disk-spool, WAL, etc.) and the recovery behavior on restart.
- Tag every event with at least: source identifier, ingestion timestamp, and the original raw payload.

3.2 Transport

Move events from collector(s) to the processor. You may use any open-source transport (gRPC, HTTP, Kafka, NATS, Redis Streams, etc.) or roll your own but justify the choice.

Mandatory properties:

- At-least-once delivery end to end. Document where and how duplicates can occur and how a downstream consumer would deduplicate if needed.
- Backpressure: if the processor or sink is slow, the collector must not silently drop events. Clearly describe the policy (block, spool, shed, sample) and why.
- Batching with a configurable size and flush interval.
- Graceful shutdown that drains in-flight batches with a bounded timeout.

3.3 Processor

A service that consumes from the transport and applies a small but pluggable processing pipeline:

- Parsers: JSON, syslog (RFC 5424), and one regex/grok-style parser. The parser must be selectable per source/route, with a sensible fallback for unparseable input.
- Enrichment: at minimum, normalize timestamps to UTC, derive severity from raw fields, and add a stable event_id.
- Routing rules: a config-driven way to drop, sample, or redact events (e.g., redact fields matching a pattern; drop debug events from a noisy host).
- Dead-letter handling: malformed or unroutable events must be captured (separate sink/table/file) rather than crashing the pipeline.

3.4 Storage/Query Sink

Persist processed events into at least one queryable sink. PostgreSQL, ClickHouse, OpenSearch, SQLite, or DuckDB or another choice is acceptable, justify the choice with respect to expected query patterns, write throughput, and retention.

Provide a small read API (HTTP or gRPC) that supports at least:

- Filter by time range (required), host/source, and severity.
- Free-text substring match on the message field.
- Pagination with a stable cursor (not raw OFFSET).

Schema and index design will be reviewed; we expect deliberate choices, not a single wide table with no indexes.

4. Non-Functional Requirements

- Observability of the pipeline itself: Nice to have but not required.
- Health and readiness endpoints that distinguish 'process is up' from 'pipeline is healthy'.
- Configuration via a single config file plus environment-variable overrides. Avoid hardcoded paths, ports, and credentials.
- Reproducible local stack: docker-compose up should bring up the collector, transport (if external), processor, storage, and any UI/metrics. A make demo or equivalent is welcome.
- Throughput target: sustain at least 10,000 events/sec on a developer laptop end-to-end with bounded memory. We will not benchmark you precisely; we want to see that you measured something and can speak to your numbers.

5. Expected Deliverables

1. **Source code** in a Git repository (GitHub link, or a tarball). Code should be production-style, not a notebook or one-file prototype.
2. **README.md** with a short architecture diagram (ASCII, Mermaid, or image), how to run locally, how to run the load test, and a 'design notes' section that explicitly discusses tradeoffs and what you would do differently with another week.
3. **docker-compose.yml** (or equivalent) that brings up the full stack with one command.
4. **Tests**: unit tests for non-trivial components, plus at least one integration/end-to-end test that proves the pipeline survives a representative failure (e.g., processor restart mid-stream, sink unavailable for N seconds, malformed input).
5. **Load-test script** (a small generator is fine) and a short note with the throughput, p50/p95 latency, and resource usage you observed.
6. **Sample queries** against the storage sink demonstrating time-range, host, severity, and substring search.

6. Evaluation Criteria

You are free (expected) to use any AI coding tool, however you should be able to explain the code and design. Reviewers will weigh the following roughly in this order:

Dimension	What we look for
Architecture & boundaries	Clean separation of collector/transport/processor/sink. Interfaces that would let us swap components. Justified choices, not just defaults.
Failure handling	Backpressure policy is explicit. At-least-once is honored under restart and slow-sink scenarios. Dead-letter path actually works. Graceful shutdown drains.
Code quality	Idiomatic Go (or chosen language). Small, testable units. No goroutine leaks. Errors are wrapped with context. Configuration is sane. Logging is structured.
Storage design	Schema fits the query pattern. Indexes are deliberate. Retention/cleanup is at least considered. No N+1 queries on the read API.
Tests	Tests target risky behavior (parsing edge cases, restart behavior, backpressure), not just happy-path constructors.
Documentation	README is enough for a teammate to run, understand, and extend the system. Tradeoffs are written down honestly.

7. Assumptions and Freedoms

- **Language:** Go is recommended given your recent work and the strong ecosystem for this domain. Python is acceptable. Mixed languages are also acceptable (e.g., Go for collector and python for the processor) Rust or C++ are acceptable if you prefer; please flag this in advance.
- **Open-source libraries and tools:** Use anything permissively licensed, Kafka, NATS, Redpanda, Vector, Fluent Bit, OpenTelemetry Collector, ClickHouse, etc. Importantly, you may not use a hosted log product (Datadog, Splunk, Loki) as a black-box substitute for what we are asking you to build; using their open-source components as building blocks is fine.
- **AI/LLM components are not required.** If you want to add a small AI-driven feature (anomaly hint, natural-language query over recent logs, automatic parser inference), it is welcome as a stretch goal but should not come at the cost of the core pipeline.
- **Single-node is fine.** You may run multiple processes via docker-compose. Multi-node clustering is not required, but your design should not preclude it; explain how you would scale horizontally.
- **Security:** TLS, authn, authz are out of scope. You may state assumptions and stop there. Do, however, handle secrets sanely (env vars or a config file, never committed).
- **Data:** Use any synthetic generator you like, or sample real public datasets (Linux syslog dumps, nginx access logs, etc.). Include the generator in the repo.
- **Time:** Aim for 6-8 hours total. If you think it's going to take more time, please scope down and document the time-consuming tasks.

10. Submission and Follow-up

Send a link to the repo (or a tarball) along with a one-page write-up covering: what you built, what you cut, what you would do with another week, and one architectural decision you are least sure about.

We will follow up with a 60–minute discussion focused on your code and your design notes.

11. Notes

This problem is defined deliberately broader than the suggested 4-8 hours. You are encouraged to scope every component meaningfully to stay within the suggested time limit.

If anything in this brief is ambiguous, you are encouraged to make a reasonable assumption, document it in the README, and move on.